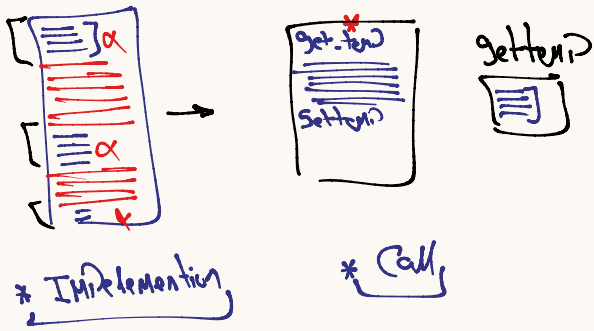


* Day 3 * Function * Array

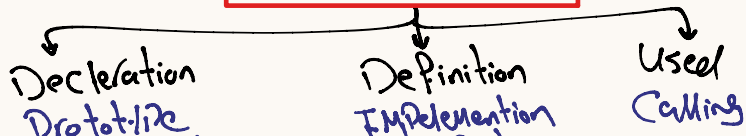
* Function → Repeat some lines of codes.
in different times.



* Function type

- ① Normal Function
- ② Function like Macro
- ③ Inline Function

Normal Function



* Declaration (Prototype).

↳ syntax:

Return type Function Name (Inputs),
 ↳ Function Return one object only;
 ↳ return keyword
 ↳ You can return any type
 ↳ int ↳ float char
 ↳ double ↳ void

Function Name Rule

- ① Descriptive
- ② Not start number
- ③ Not start symbol

Inputs:

- ↳ ① You can have void input
- ↳ ② You can have unlimited input
- ↳ ③ Input style

Data type varName → int x
 [int Add2Number(int x, int y);]

Declaration

↳ where write it !:

in Masra-C → must be write it
before main or in an header
file

File.h
 math.h
 [prototype] → Main.c
 #include "math.h"

* Definition [Implementation]

↳ write the Definition After main
or source file.

return type Function Name (Inputs)
 ↳ int x
 Implementation

* Used Calling

↳ Function No Return

function Name (values)

C code Function Print Numbers

void DisplayNo (int No); → Prototype

int main()

↳ int No = 5;

→ DisplayNo (No);

Calling

void DisplayNo (int No) → Implementation

↳ printf ("Number = %d\n", No);

C Code to Scan Number From user

int ScanDecimalNo (void);

int ScanDecimalNo (void)

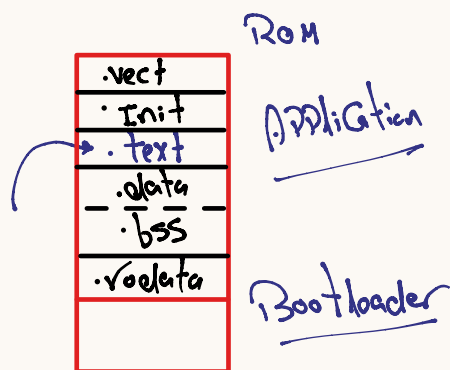
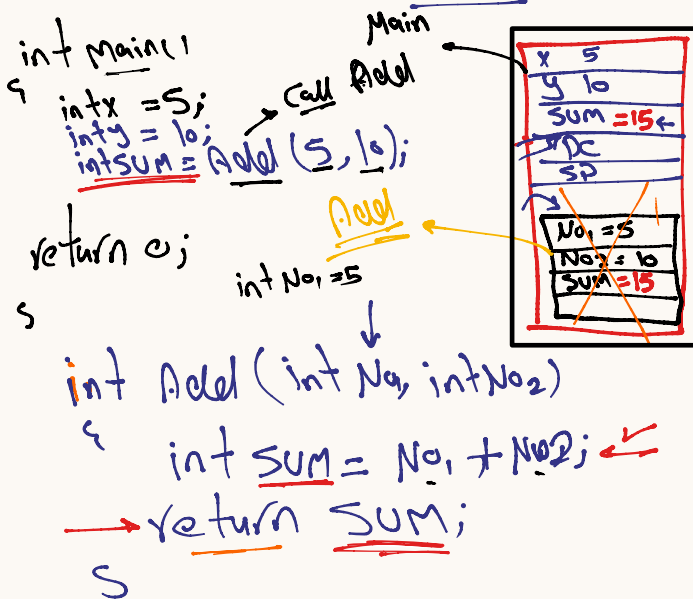
↳ int No = 0;

scanf ("%d", &No);

return No;

* How to store in Memory

- ① Implementation save in ROM → because it code
- ② Implementation store in text section
- ③ Call → execution → local variable store in RAM → stack



* what happen during Call

* Processor not jump immediatly to a new line code but it do the context switching

* Context switching meaning
 → store to some information related to processor in RAM [stack]

- * Program Counter ⇒ PC
- * Stack Pointer ⇒ SP
- * General Purpose Register ⇒

* Function Name is → Address of Implementation

Add(5, 7);
 → jump to function

* Normal Function

about memory → save memory, use small bytes,
 → variable remove when function finished

about time execution → slow execution take along time execute

* Re Cap

- ① Function meaning & why
- ② Function type
- ③ Normal

→ Declaration / Definition / Call
 → How create Header file & Source file
 → Memory Handling
 → Call scenario

[2] Function like macro

*style

#define Calling(inputs) expression

#define Add(x, y) x + y

int main()

{ int Result = Add(5, 7);

}

Preprocessor

int main()

{ int Result = 5 + 7;

* Multiline Function like Macro ()

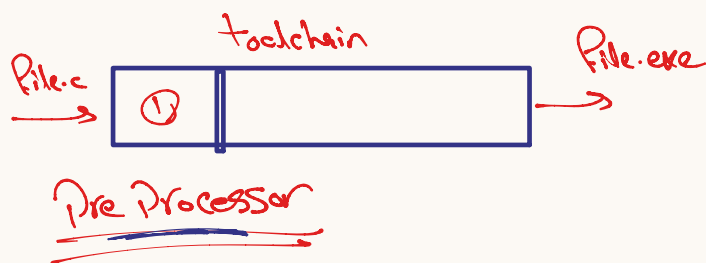
#define welcomeMessage() printf();
printf();
printf()

* Function like Macro don't have memory
but the effect have a lot of size

Function like Macro		<u>total Normal</u>
Add(5, 7) → ②		↳ 2 Byte
Add(10, 10) → ②		<u>total Macro</u>
Add(20, 30) → ③		↳ 6 Byte

Execution
Fast

Size
take a large
Size



<u>Normal Function</u>	<u>Macro</u>
Slow time	Fast time
Save Memory	take Memory
CPU (Processor) exelution	Preprocessor <u>toolchain</u>

Inline Function

* function like normal from writing

* Request for Compiler

if this is calling don't jump → take the code copy and past in the same line

→ Compiler → Function if Calling

* don't jump

* take the code and copy in the calling line

	Normal	Macro	Inline
type safety	✓ compile check on type	X just text replacement	✓ Compiler check on data
Performance	slow [Context]	very Fast execution in call line b/c pre processor	very fast execution in call line by compiler
Code size	Small size	large size	large size

Inline Function

→ writing way → like normal function

→ How Handle → like Macro

Request compile ^{Inline} Preprocessor

→ Cancel jump to Address

Memory Handling :- Inline Function

→ Ram → low Consumption
→ No Context switching

→ ROM → High Consumption

Function → (20) text code
Call → 10 200 line

→ time execution

→ fast execution

Introduction Array

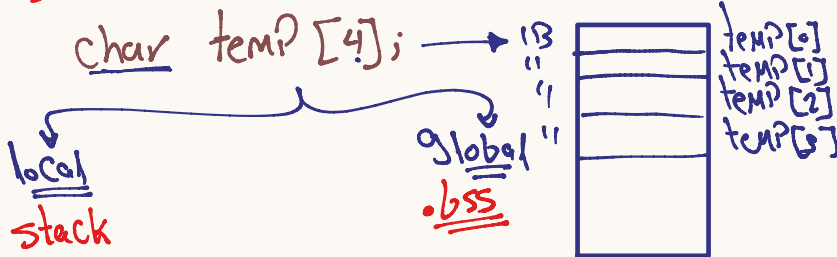
* Array is Data structure type.

↳ Some Variable Stored in Memory

- ↳ ① under same category
- ↳ ② same Data type
- ↳ ③ Fixed size
- ↳ ④ all variable store on order in Memory
- ↳ ⑤ start index from zero until size - 1

* Array Declaration

Data type ArrayName [Fixed size];



* Array Definition

Data type ArrayName [Fixed size] = { Init0, Init1, ... InitSize-1 };

ex1. char No[3] = { 30, 20, 40 };

30	No[0]
20	No[1]
40	No[2]

ex2. char test[4] = { 3, 2, 1, 0 };

3	test[0]
2	test[1]
0	test[2]
0	

ex3. char speed[1000] = { 909 };

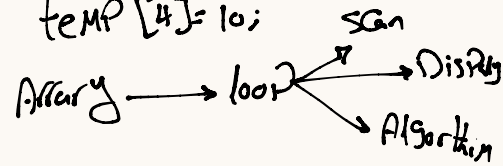


test = {
}; only Definition

How Access it? (Modify)

char temp[5];

temp[0] = 3;
temp[1] = 3;
temp[2] = 0;
temp[3] = 5;
temp[4] = 10;



```
for(count=0; count<Size; count++)
{
    printf("Enter the number: ");
    scanf("%d", &temp[count]);
}
```

```
for(count=0; count<Size; count++)
{
    printf("Number=%d", temp[count]);
}
```

char array[5]
= { 'A', 'B', 'C', 'D', 'E' };

* Array Name is Address for first location in Array

int arr[4] = { 909 }

0x00:0x03	0	arr[0]
0x04:0x07	0	arr[1]
0x08:0x12	0	arr[2]
	0	arr[3]
	0x00	

arr
Can't modify this location